

# GhostNet

VIVA SHEET :: DESIGN AND ANALYSIS OF ALGORITHMS

GhostNet is hiring without names. A company posts a real problem instead of a job advert. Anyone can send an answer under a fake name called a ghost id. The system scores every answer and puts them in order. Only then does it show who wrote the winning one.

The website is not the interesting part. The interesting part is this: how do you put fifty answers in a fair order when all you have is the work itself? That takes nine algorithms. Each of us owns one.

## HOW THE SYSTEM ACTUALLY RUNS

1. A company posts a challenge. It is saved in the challenges table.
2. People send answers. Each one gets a ghost id. The real name is saved in a column that the scoring never reads.
3. Someone clicks Rank. That one request runs five algorithms over every answer and saves the scores.
4. Merge sort puts the rows in order. The table appears, one column per algorithm.
5. The company clicks Reveal. The real name shows for the first time, and the win is hashed into the proof chain.

## THE STACK

|                   |                                                                 |
|-------------------|-----------------------------------------------------------------|
| <b>Backend</b>    | Python 3.12, FastAPI, Uvicorn, Pydantic                         |
| <b>Database</b>   | SQLite on a laptop, PostgreSQL 16 when deployed, one set of SQL |
| <b>Frontend</b>   | Next.js 15, React 19, TypeScript, plain CSS                     |
| <b>Testing</b>    | pytest, 12 tests, and none of them needs the database           |
| <b>Automation</b> | GitHub Actions, three jobs, runs on every push                  |
| <b>Algorithms</b> | All written by hand. No library does the work for us            |

## HOW A SUBMISSION IS SCORED

$$\text{final} = (0.45 \times \text{relevance} + 0.25 \times \text{quality} + 0.30 \times \text{structure}) \times (1 - \text{plagiarism})$$

Plagiarism multiplies instead of adding, so copied work cannot win on the strength of the other three columns. The weights sit at the top of algorithms/ranker.py where anyone can read them.

## WHO OWNS WHAT

|    |                               |                                           |
|----|-------------------------------|-------------------------------------------|
| 1  | <b>Algorithm Engineer</b>     | Cosine similarity with tf idf             |
| 2  | <b>Backend Developer</b>      | Levenshtein edit distance                 |
| 3  | <b>Frontend Developer</b>     | Merge sort, written twice                 |
| 4  | <b>Database Engineer</b>      | Rabin Karp with a rolling hash            |
| 5  | <b>QA Engineer</b>            | Suffix array and longest common substring |
| 6  | <b>DevOps Engineer</b>        | Cyclomatic complexity                     |
| 7  | <b>Blockchain Developer</b>   | Merkle tree and a SHA 256 chain           |
| 8  | <b>Product Manager</b>        | Gale Shapley stable matching              |
| 9  | <b>UI and UX Designer</b>     | Greedy interval scheduling                |
| 10 | <b>Docs and Research Lead</b> | Measuring the cost of all nine            |

## RULES FOR THE ROOM

- Know your own algorithm well. The big O, why it is that, and one tiny example you can do on the board.
- Point at your file. Everyone owns at least one and it is listed on your page.
- Be ready to say what you built, not only what your algorithm does. Half the marks are in the first one.
- If you do not know, say you do not know and say how you would find out. That is better than guessing.
- Do not call your own work simple. Say what it costs instead.

# Algorithm Engineer

MEMBER 1 :: COSINE SIMILARITY WITH TF IDF

I wrote the relevance score. It is the biggest part of the final mark. It answers one question: how close is this answer to what the company asked for?

## WHAT I BUILT

- A function that breaks text into words. It removes punctuation and makes everything lowercase.
- Term frequency, or tf: how many times each word appears, divided by the length of the text.
- Inverse document frequency, or idf: how rare a word is across all the answers. Common words lose their weight.
- The vector builder and the cosine function. Both are about ten lines.
- One function, `relevance_scores`, that takes the problem and all the answers and gives back a score for each.

## THE FILES I OWN

|                             |                           |
|-----------------------------|---------------------------|
| backend/algorithms/tfidf.py | all of it, about 70 lines |
|-----------------------------|---------------------------|

## TOOLS I USED, AND WHY THOSE

Python and nothing else. I import math for log and sqrt. I did not use numpy or scikit learn. A library would have done the work for me, and this is the work the course is marking.

## SOMETHING THAT WENT WRONG

|                                                                                                                                                                                                                                                          |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| My first version just counted words. Long answers won, because they had more words. I fixed it by dividing by the text length. Then common words like the and and took over every comparison. That is the exact problem idf solves, so I added idf next. |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

## HOW MY ALGORITHM WORKS

1. Break each text into words. Count each word. Divide by the number of words in that text. This is tf.
2. Count how many texts each word appears in. Rare words get a high weight. Words in everything get almost zero. This is idf.
3. Multiply tf by idf. Each text is now a list of numbers, one number per word. We call that a vector.
4. Score is the dot product of two vectors divided by both their lengths. That is the cosine of the angle between them. It is always between 0 and 1.

## COMPLEXITY

$O(N \times L)$  to build all the vectors.  $N$  is how many answers there are,  $L$  is how long each one is. Comparing two vectors is  $O(V)$ , where  $V$  is the number of words they share.

## HOW MY PART CONNECTS TO EVERYBODY ELSE

`ranker.py` calls my function once with all the answers together. It has to be together, because idf needs to see the whole set. My number becomes the Relevance column and is 45 percent of the score.

## LIKELY QUESTIONS

|                                                                 |                                                                                                                                                                                                                 |
|-----------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Why cosine and not straight line distance?                      | Straight line distance gets bigger when a text is longer. So a long answer would look far away from a short problem, even if it says the same thing. Cosine only looks at the angle, so length does not matter. |
| Why do you add 1 inside the log?                                | It is called smoothing. A word that is in every text would give log of 1, which is 0. That would delete the word. Adding 1 keeps a small weight instead.                                                        |
| What is weak about this method?                                 | It only looks at which words appear, not their order. So it cannot tell a right answer from the same words in the wrong order. That is why it is 45 percent of the score and not all of it.                     |
| Why work out idf over all the answers instead of one at a time? | idf describes the whole group, not one text. If you did it for one text alone the answer would always be the same number, so it would do nothing.                                                               |

## SHOW THIS ON SCREEN

The Relevance column in the ranking table. Ghost 001 is highest because it uses the same words as the problem. Then open `tfidf.py` and point at the cosine function.

# Backend Developer

MEMBER 2 :: LEVENSHTein EDIT DISTANCE

I built the API. Everything else talks to it. My algorithm is edit distance. It counts the smallest number of letter changes needed to turn one text into another.

## WHAT I BUILT

- Ten endpoints. List and create challenges, list and add answers, rank, read results, reveal, the proof chain, matching and scheduling.
- Pydantic models. A bad request is rejected with a clear message before any of our code runs.
- The lifespan handler. It opens the database pool when the server starts and closes it when the server stops.
- CORS, so the frontend on port 3000 is allowed to call port 8000.
- Edit distance, using only two rows instead of a full table.

## THE FILES I OWN

|                                   |                                    |
|-----------------------------------|------------------------------------|
| backend/main.py                   | the API, about 200 lines, no logic |
| backend/algorithms/levenshtein.py | edit distance and the score        |

## TOOLS I USED, AND WHY THOSE

FastAPI, because it writes the API documentation for us from the type hints. That means /docs is never out of date. Uvicorn runs it. Pydantic checks the input. psycopg 3 talks to PostgreSQL.

## SOMETHING THAT WENT WRONG

We started on SQLite and moved to PostgreSQL. Every placeholder changed from a question mark to percent s. PostgreSQL has no lastrowid, so every insert that needed an id got a RETURNING clause. The old startup hook was also removed from FastAPI, so it became a lifespan handler.

## HOW MY ALGORITHM WORKS

1. Make a table. Cell  $i, j$  holds the distance between the first  $i$  letters of one word and the first  $j$  letters of the other.
2. Fill each cell with the smallest of three choices: insert a letter, delete a letter, or replace a letter. Replace is free if the two letters already match.
3. The answer is the bottom right cell.
4. Row  $i$  only needs row  $i$  minus 1. So we keep two rows and throw the rest away.

## COMPLEXITY

$O(m \times n)$  time. Every cell is filled once and there are  $m$  times  $n$  cells.  $O(n)$  memory, because we only keep two rows at a time.

## HOW MY PART CONNECTS TO EVERYBODY ELSE

Every screen calls my routes. The rank route is the important one. It loads the answers, gives them to ranker.py, saves the scores, and sends them back. All the thinking happens in the algorithms folder, not in my file.

## LIKELY QUESTIONS

|                                                   |                                                                                                                                                                                                  |
|---------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Show kitten turning into sitting.                 | Replace k with s. Replace e with i. Add g at the end. Three changes. There is a test that checks this exact example.                                                                             |
| Why dynamic programming and not simple recursion? | Recursion solves the same small problem again and again. That is exponential time. The table solves each small problem once, which gives $m$ times $n$ .                                         |
| Why is your API so thin?                          | Each route checks the input, calls one module, and returns JSON. Because there is no logic in it, all twelve tests run with no server and no database.                                           |
| What if two people submit at the same moment?     | Each request takes its own connection from the pool and its own transaction, so they do not clash. The ghost id is unique in the database, so a repeat would be rejected, not silently accepted. |

## SHOW THIS ON SCREEN

Open <http://localhost:8000/docs>. FastAPI made that page from the type hints, and the examiner can call any endpoint from it. Then the Quality column in the ranking table.

# Frontend Developer

MEMBER 3 :: MERGE SORT, WRITTEN TWICE

I built every screen. I also own the sort that turns a list of scores into a leaderboard. It runs on the server for the first ranking, and again in the browser when you click a column.

## WHAT I BUILT

- Five screens. The challenge list, one challenge with submit and ranking, matching, schedule, and the proof chain.
- A typed API client, so every call to the backend goes through one file and errors are handled in one place.
- types.ts, which copies the shapes from the backend. If the backend changes, my editor shows a red line instead of the page going blank.
- The ranking table, where each column header also names the algorithm that made that number.
- Merge sort in TypeScript, used when you click a header.

## THE FILES I OWN

|                               |                               |
|-------------------------------|-------------------------------|
| frontend/src/app/             | the five screens              |
| frontend/src/lib/mergeSort.ts | the algorithm                 |
| frontend/src/lib/api.ts       | every call to the backend     |
| frontend/src/lib/types.ts     | the shapes the API sends back |

## TOOLS I USED, AND WHY THOSE

Next.js and React for the pages. TypeScript, because it catches a wrong field name while I type instead of during the demo. Plain CSS from our own design system. No component library and no chart library.

## SOMETHING THAT WENT WRONG

Next.js 15 changed how route parameters arrive. They come as a promise now, not a plain object, and the challenge page broke. I unwrap it with the React use hook. Separately, my first version used the built in sort. It is not guaranteed stable, so rows with equal scores jumped around on every click. Writing merge sort fixed that, and it is also the part the course marks.

## HOW MY ALGORITHM WORKS

1. Cut the list in half. Keep cutting until every piece has one item. One item is already in order.
2. Join two ordered halves back together. Walk both with one pointer each and always take the bigger one first.
3. There are  $\log n$  levels of cutting, and each level touches all  $n$  items. That is where  $n \log n$  comes from.
4. If two scores are equal, take the left one first. This is what makes the sort stable.

## COMPLEXITY

$O(n \log n)$  every time. Best case, average case and worst case are all the same, which is the main reason we chose it. It needs  $O(n)$  extra space to hold the joined list.

## HOW MY PART CONNECTS TO EVERYBODY ELSE

The backend sends rows already in order. Clicking a header sorts them again in the browser with no network call, so the table reorders instantly. types.ts is the agreement between my code and the backend developer's code.

## LIKELY QUESTIONS

|                                                |                                                                                                                                                                 |
|------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Why merge sort and not quicksort?              | Quicksort can be $O(n^2)$ in the worst case, and it is not stable. Rows with the same score would swap places every time the table redraws, which looks broken. |
| Why does stable matter here?                   | Equal scores must keep the order the answers arrived in. If they do not, the ranking looks random and we cannot defend it.                                      |
| Why write the same algorithm in two languages? | To show that the cost belongs to the algorithm, not the language. Both versions are $n \log n$ .                                                                |
| Why not just use the built in sort?            | Because the algorithm is what the course is marking. The built in sort would hide it, and it is not guaranteed stable.                                          |

## SHOW THIS ON SCREEN

Click any column header in the ranking table. It reorders with no network request. Then show mergeSort.ts next to merge\_sort.py.

# Database Engineer

MEMBER 4 :: RABIN KARP WITH A ROLLING HASH

I designed the database, and I own the copying check. Hiding names is a database decision before it is anything else. My algorithm is what stops copied work from winning.

## WHAT I BUILT

- The three tables. challenges, submissions and results, with real types, foreign keys, and cascade delete.
- One db.py that speaks to both SQLite and PostgreSQL. The rest of the project writes one style of SQL and this file translates.
- Two indexes. PostgreSQL does not index a foreign key by itself, and we filter on both of ours all the time.
- A unique rule on challenge id and ghost id in results. Running the ranking twice now updates the scores instead of adding a second copy.
- The connection pool, and the two helpers query and execute that the whole API uses.
- Rabin Karp, and the chunking that turns it into a copying percentage.

## THE FILES I OWN

|                                  |                                  |
|----------------------------------|----------------------------------|
| backend/db.py                    | tables, pool, query and execute  |
| backend/algorithms/rabin_karp.py | the search and the overlap score |

## TOOLS I USED, AND WHY THOSE

SQLite on a laptop, so nobody has to install anything to run the project. PostgreSQL 16 on the deployed site, for real types and real rules, reached with psycopg 3 and a connection pool. Rows come back as dictionaries either way, which keeps the API short.

## SOMETHING THAT WENT WRONG

Running the ranking twice added a second set of rows, and the table showed every answer twice. I added a unique rule on challenge id and ghost id. Now the database makes it impossible instead of us hoping the code remembers to clean up.

## HOW MY ALGORITHM WORKS

1. Treat a run of letters as one big number in base 256, kept small using modulo a large prime.
2. Hash the pattern once. Hash the first window of the text once.
3. Slide the window by one letter. Take away the letter that left, multiply by the base, add the letter that arrived. That is one step, not m steps.
4. If two hashes match, still compare the real letters. Two different texts can share a hash by accident.

## COMPLEXITY

$O(n + m)$  on average. The worst case is  $O(n \times m)$ , which happens if every window has the same hash as the pattern by accident and we have to check the letters every time.

## HOW MY PART CONNECTS TO EVERYBODY ELSE

The copying score multiplies the final mark instead of being added to it. So a copied answer cannot win by being good in the other columns. Every other member's data goes through db.py.

## LIKELY QUESTIONS

|                                                      |                                                                                                                                                                                                                                                                                                                   |
|------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Why is a matching hash not enough on its own?</b> | Two different texts can give the same hash. That is called a collision. Checking the real letters turns a maybe into a yes.                                                                                                                                                                                       |
| <b>Why modulo a large prime?</b>                     | It keeps the numbers small enough for the computer to handle, and it spreads the hashes out so collisions are rare.                                                                                                                                                                                               |
| <b>How do the tables hide the name?</b>              | The real name is in a column that the scoring queries never ask for. So nobody has to remember the rule. The name is simply not in the SQL.                                                                                                                                                                       |
| <b>Why those two indexes?</b>                        | PostgreSQL indexes a primary key by itself but not a foreign key. We filter on both foreign keys in nearly every query. Without the indexes, every read has to scan the whole table.                                                                                                                              |
| <b>What does the connection pool save?</b>           | Opening a connection and logging in takes longer than most of our queries. The pool keeps a few open and lends them out, so we pay that cost once at startup instead of on every request. SQLite needs no pool, because it is a file and not a server.                                                            |
| <b>Two databases sounds risky. Why do it?</b>        | It is risky, and we know the risk: something can work on a laptop and break when deployed. We handle it two ways. All the differences live in one file, db.py, so there is one place to look. And CI runs the whole API against a real PostgreSQL container on every push, so the two cannot quietly drift apart. |

## SHOW THIS ON SCREEN

The Copied column. Ghost 003 is at 87 percent and comes last, even though its other scores are normal. Then open db.py and show the tables and the unique rule.

# QA Engineer

MEMBER 5 :: SUFFIX ARRAY AND LONGEST COMMON SUBSTRING

I test what the team writes. My algorithm finds the exact piece of text that was copied. Rabin Karp says how much was copied. Mine says what was copied, word for word.

## WHAT I BUILT

- Twelve tests, one per algorithm. Every input is small enough to check by hand on the board.
- A test that proves changing an old win breaks every hash after it. That is the claim the whole proof chain rests on.
- A test that proves copied work scores 1.0 and unrelated work scores under 0.2.
- The suffix array, and the search for the longest shared piece of text.
- The test run that GitHub does on every push.

## THE FILES I OWN

|                                    |                            |
|------------------------------------|----------------------------|
| backend/tests/test_algorithms.py   | twelve tests, no database  |
| backend/algorithms/suffix_array.py | suffix sort and the search |

## TOOLS I USED, AND WHY THOSE

pytest. A test is just a function with an assert in it, so there is nothing extra to learn. The tests import the algorithms directly, so they finish in under a second with no server and no database.

## SOMETHING THAT WENT WRONG

I found a real bug. An essay with no code was getting a perfect structure mark and coming first. The reason was that cyclomatic complexity only punished code that was too complicated. I reported it. Member 6 changed the score to a healthy range of 3 to 10, so an answer with no branching loses marks too. There is now a test that fails if anyone undoes it.

## HOW MY ALGORITHM WORKS

1. Join the two texts with a separator character that is in neither of them.
2. Take every suffix of the joined text. A suffix is what is left from a point to the end. Sort them all.
3. Similar text now sits next to each other. Walk the sorted list and compare neighbours, but only pairs that came from different sides of the separator.
4. The longest shared start among those pairs is the longest copied piece.

## COMPLEXITY

$O(n \log n \times \log n)$  to sort the suffixes. There are  $n \log n$  comparisons and each one can compare up to  $n$  letters. Then  $O(n)$  to walk the sorted list once.

## HOW MY PART CONNECTS TO EVERYBODY ELSE

The piece of text my algorithm finds is printed in the last column of the ranking table. So the copying percentage is not something the examiner has to take our word for.

## LIKELY QUESTIONS

|                                          |                                                                                                                                                                                                            |
|------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Why sort the suffixes at all?            | Sorting puts texts that start the same way next to each other. Then the longest shared piece is always between two neighbours, so one walk through the list finds it.                                      |
| Why do you need the separator character? | Without it, a match could start in the first text and run into the second. That would report a piece of text that is in neither of them.                                                                   |
| Why are your test inputs so small?       | So every test can be checked by hand. If the examiner asks why a number is right, the answer is readable in the test instead of hidden in a debugger.                                                      |
| What would you improve?                  | My suffix sort compares whole strings, which adds a factor of $n$ . There are faster methods called DC3 and SA IS that give true $O(n)$ , but they are much harder to read and this is a teaching project. |

## SHOW THIS ON SCREEN

Run `python -m pytest tests -v`. Twelve tests, under a second, no database needed. Then the last column of the ranking table.

# DevOps Engineer

MEMBER 6 :: CYCLOMATIC COMPLEXITY

I make the project run the same way on ten different laptops, and I get it checked automatically on every push. My algorithm counts how tangled a piece of code is.

## WHAT I BUILT

- setup\_db.py. It creates the user, the database and the tables, and prints a readable message when PostgreSQL is not running.
- Settings through the DATABASE\_URL environment variable. The same code runs on a laptop, on a free cloud database, and in CI with no edits.
- A GitHub Actions pipeline with three jobs: algorithms, api and frontend.
- The health endpoint, which CI and any hosting service can check.
- Cyclomatic complexity, and the scoring range built on top of it.

## THE FILES I OWN

|                                  |                            |
|----------------------------------|----------------------------|
| backend/setup_db.py              | one command database setup |
| .github/workflows/ci.yml         | three jobs on every push   |
| backend/algorithms/complexity.py | the measure                |

## TOOLS I USED, AND WHY THOSE

GitHub Actions for the automatic checks. PostgreSQL installed the normal way instead of through Docker, because the team is on Windows laptops and one more heavy tool was not worth it. The CI job gets its own database from GitHub, so nobody installs it.

## SOMETHING THAT WENT WRONG

My first version gave full marks to anything under ten. So an essay with no code scored 1.0 and nearly won. QA found it. I changed it to a healthy range of 3 to 10. Too little branching means nothing was really built. Too much means it is a mess. Both ends now lose marks.

## HOW MY ALGORITHM WORKS

1. Picture the code as a graph. Each statement is a node. Each jump between statements is an edge.
2. The number of separate paths through it is E minus N plus 2. E is edges and N is nodes.
3. For one function this becomes simpler: count the decision points and add 1. Every branch adds exactly one path.
4. We count if, elif, for, while, case, catch, except, and, or, and the question mark operator.

## COMPLEXITY

O(n) over the letters of the code, in one pass.

## HOW MY PART CONNECTS TO EVERYBODY ELSE

My score becomes the Structure column, worth 30 percent. My CI pipeline runs the QA tests and the frontend build, so a broken algorithm is caught before the demo, not during it.

## LIKELY QUESTIONS

|                                                                 |                                                                                                                                                                                                         |
|-----------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Why punish low complexity as well as high?</b>               | A score of 1 or 2 means there is almost no branching. For a coding challenge that means nothing was really built. Without the lower limit, an essay beats working code. We saw that happen.             |
| <b>Why is 10 the upper limit?</b>                               | It is the number the industry normally uses for one function. Above it there are too many paths to test and it gets hard to read.                                                                       |
| <b>Why count keywords instead of properly reading the code?</b> | A proper reader only works for one language, and answers can be in any language. Counting keywords works for all of them. We are honest that it is an estimate.                                         |
| <b>Why did you drop Docker?</b>                                 | It is one more tool for ten people to install on Windows laptops, and all it was doing was starting PostgreSQL. A setup script does the same job in code anyone on the team can read.                   |
| <b>What does CI actually catch?</b>                             | Three things. The algorithm tests. A full check that the copied answer still comes last. And the frontend type check and build. If someone changes the scoring and copied work climbs, the build fails. |

## SHOW THIS ON SCREEN

The Structure column, with the path count under each score. Then ci.yml, and the line in the api job that says the copied answer must still come last.

# Blockchain Developer

MEMBER 7 :: MERKLE TREE AND A SHA 256 CHAIN

A win has to be provable without trusting us. I built the Skill Proof record. Every win is hashed into a tree, and anyone can check one win using only a few hashes instead of the whole list.

## WHAT I BUILT

- The Merkle tree builder. It hashes the wins, then joins them in pairs going up until one hash is left.
- Proof making. It returns only the partner hashes on the path from one win up to the top, and remembers which side each one is on.
- Checking. It rebuilds the top hash from one win and its path.
- The hash chain, where every block carries the hash of the block before it.
- The proof chain screen, which prints the previous hash right above each block hash so you can see the link.

## THE FILES I OWN

|                              |                              |
|------------------------------|------------------------------|
| backend/algorithms/merkle.py | tree, proof, check and chain |
| frontend/src/app/chain/      | the screen                   |

## TOOLS I USED, AND WHY THOSE

hashlib for SHA 256, and nothing else. The tree, the proof, the checking and the chain are all written by hand. There is no network and no mining, and that is on purpose.

## SOMETHING THAT WENT WRONG

If there is an odd number of wins, one node at some level has no partner. I pair that node with itself, which is what most real systems do. I also had to record whether each partner is on the left or the right. Hashing them in the wrong order gives a different root and the check fails.

## HOW MY ALGORITHM WORKS

1. Hash every win. These hashes are the bottom row of the tree, called the leaves.
2. Hash each pair of neighbours together to make the row above. Repeat until one hash is left. That one is the root.
3. To prove one win is in the tree, hand over the partner hash at each level. That is  $\log n$  hashes, not  $n$ .
4. The checker rebuilds the root from the win and the partners. If it matches the published root, the win is real.

## COMPLEXITY

$O(n)$  to build the tree, because each node is hashed once.  $O(\log n)$  to make or check one proof, because the tree has  $\log n$  rows.

## HOW MY PART CONNECTS TO EVERYBODY ELSE

The reveal endpoint returns the root, the winning leaf, and its proof. The chain endpoint links all revealed wins into blocks. This is what turns a ranking into a credential you can carry.

## LIKELY QUESTIONS

|                                                  |                                                                                                                                                                                                 |
|--------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Why a tree and not just a list of hashes?</b> | A list needs all $n$ hashes to check one entry. A tree needs $\log n$ . With a thousand wins that is ten hashes instead of a thousand.                                                          |
| <b>What stops someone changing an old win?</b>   | Every block holds the hash of the block before it. Change block 0 and its hash changes, which changes block 1, and so on all the way down. There is a test that proves this.                    |
| <b>Is this a real blockchain?</b>                | No, and that is on purpose. A network needs mining and many computers. That takes weeks and shows nothing extra. The part that matters is that changes are visible, and the tree gives us that. |
| <b>Why SHA 256 and not something faster?</b>     | If someone could find a different input with the same hash, they could fake a win. SHA 256 makes that practically impossible. Something like MD5 would not.                                     |

## SHOW THIS ON SCREEN

Reveal a winner, then read the root, the leaf and the proof path. Then the Proof chain page, where each block shows the previous hash above its own.

# Product Manager

MEMBER 8 :: GALE SHAPLEY STABLE MATCHING

I decided what we would build, and just as importantly what we would not. My algorithm handles the last step. Several winners, several companies, and a pairing that nobody wants to walk away from.

## WHAT I BUILT

- The scope. One claim, tested end to end: work can be put in order without knowing whose work it is.
- A four sprint plan, showing what is done and what is still open.
- A decisions table that records every cut and the reason for it.
- The preference lists for both sides of the match.
- The proposal log, which records every offer, rejection and swap. The log is the explanation.

## THE FILES I OWN

|                                    |                                     |
|------------------------------------|-------------------------------------|
| docs/ROADMAP.md                    | sprints, decisions, and what we cut |
| backend/algorithms/gale_shapley.py | the matching                        |

## TOOLS I USED, AND WHY THOSE

Markdown for the roadmap, so it sits in the repository next to the code and changes with it, instead of living in a chat nobody can find later.

## SOMETHING THAT WENT WRONG

My first version only returned the final pairs, and nobody could tell whether it was right. Adding the proposal log turned it from a black box into something you can follow line by line. That is also what makes it possible to present.

## HOW MY ALGORITHM WORKS

1. Every free candidate makes an offer to the best company they have not asked yet.
2. A company with no offer holds the one it just got.
3. A company that already has an offer keeps whichever candidate it likes more and lets the other one go.
4. Anyone let go goes back in the pool and offers further down their list. It always ends, because nobody asks the same company twice.

## COMPLEXITY

$O(n^2)$ . There are  $n$  candidates and each asks at most  $n$  companies, so at most  $n^2$  offers, and each one is quick. It always finishes, and the answer is always stable.

## HOW MY PART CONNECTS TO EVERYBODY ELSE

Ranking picks the best answer. Matching handles what happens when several winners and several companies all have preferences. The Matching screen shows the log so you can argue with the result.

## LIKELY QUESTIONS

|                                             |                                                                                                                                                                                             |
|---------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| What does stable mean here?                 | There is no candidate and company who both like each other more than what they got. If there were, those two would leave and pair up on their own, and the whole matching would fall apart. |
| Does it matter which side makes the offers? | Yes. The side that offers gets the best stable result it can. The side that receives gets the worst. We let candidates offer, which fits the whole point of the product.                    |
| Why did you cut login and payments?         | Neither one proves the claim we are making. Both are in the out of scope list in the roadmap with the reason written down.                                                                  |
| What would you build next?                  | Company accounts, so a real company can post a challenge without us adding it by hand. Everything else on the cut list is further away.                                                     |

## SHOW THIS ON SCREEN

The Matching page. Click Run matching and read the log line by line. Then the decisions table in ROADMAP.md.

# UI and UX Designer

MEMBER 9 :: GREEDY INTERVAL SCHEDULING

I decided how the product looks and how it reads. There is one rule: no screen ever shows a name next to a score. My algorithm picks which challenge time slots can run without clashing.

## WHAT I BUILT

- The screen order. Challenges, then one challenge, then the ranking, then the reveal. That is also the order we walk the demo.
- The design system for this project, built on tokens, so the whole app can be restyled from one block of CSS.
- The ranking table layout, where every column header also names the algorithm that made the number.
- The timeline on the schedule screen. It draws the time slots as bars instead of listing numbers.
- The accessibility rules. Colour only where it carries meaning, aria current on the active menu item, and a real label on every input.

## THE FILES I OWN

|                                  |                                    |
|----------------------------------|------------------------------------|
| docs/DESIGN.md                   | the rules and the reasons          |
| frontend/src/app/ghostnet.css    | what the base system did not cover |
| backend/algorithms/scheduling.py | the greedy choice                  |

## TOOLS I USED, AND WHY THOSE

Plain CSS with variables. No framework and no component library. The look is called brutalist. It is black and white, and shapes come from thick black borders instead of from colour.

## SOMETHING THAT WENT WRONG

At first the schedule was a list of numbers and nobody could see why a slot had been dropped. Drawing the slots as bars on one timeline made the overlap obvious. The greedy choice became something you can see instead of something we claim.

## HOW MY ALGORITHM WORKS

1. Sort every time slot by when it ends, earliest first.
2. Take the first one.
3. Go through the rest. Take a slot only if it starts at or after the last one you took ended.
4. That is the greedy choice. Finishing early leaves the most room for everything after it.

## COMPLEXITY

$O(n \log n)$ . Walking the list is only  $O(n)$ , so the sorting is the whole cost of this algorithm.

## HOW MY PART CONNECTS TO EVERYBODY ELSE

Names are hidden in two places, and one of them is mine. The API never asks for the name column, and the TypeScript type says the name can be empty. So the compiler complains if a screen tries to show it before the reveal.

## LIKELY QUESTIONS

|                                                                    |                                                                                                                                                                                                                                |
|--------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Why sort by end time and not start time, or shortest first?</b> | Sorting by start time can pick one long slot that blocks everything else. Shortest first can pick a small slot sitting across the middle of two others. Sorting by end time is the only one of the three we can prove is best. |
| <b>How do you prove it is best?</b>                                | Take any best possible schedule. Swap its first slot for the one that ends earliest. The schedule is still valid and still the same size. So the greedy choice is never worse.                                                 |
| <b>Where does the time go?</b>                                     | The walk is $O(n)$ and the sort is $O(n \log n)$ , so the sort is what counts.                                                                                                                                                 |
| <b>Why hide names in the design and not just in the API?</b>       | Because hiding names is the product, not a rule. If a screen shows a name, the screen is wrong. That is why the type says the name can be empty.                                                                               |

## SHOW THIS ON SCREEN

The Schedule page. Two slots kept in green, one dropped in grey, and you can see the overlap on the timeline.

# Docs and Research Lead

MEMBER 10 :: MEASURING THE COST OF ALL NINE

Everyone else claims a big O. My job is to run the code and show whether the claim is true, and to write the whole thing down.

## WHAT I BUILT

- A benchmark that runs every algorithm at four growing input sizes and prints the measured milliseconds next to the claimed cost.
- The README, including the seven step demo we follow on presentation day.
- This viva sheet. The words come from a separate file so any member can fix their own page.
- The notes under the benchmark table, including the places where the measurement and the theory do not agree.

## THE FILES I OWN

|                      |                                  |
|----------------------|----------------------------------|
| backend/benchmark.py | the measuring                    |
| README.md            | how to run it and how to demo it |
| docs/viva_content.py | the words on this sheet          |

## TOOLS I USED, AND WHY THOSE

reportlab to build this PDF. Markdown for the report, so it lives in the repository. time.perf\_counter for timing, because it only goes forwards and does not jump if the system clock changes.

## SOMETHING THAT WENT WRONG

The Merkle proof is described as  $O(\log n)$  but it measures as  $O(n)$ . Our version rebuilds the whole tree before walking it, so the time is linear even though the proof it gives back is still only  $\log n$  hashes. Instead of quietly relabelling it, the benchmark prints the note.

## HOW MY ALGORITHM WORKS

1. Make random input at four sizes: 100, 200, 400 and 800.
2. Time every algorithm at each size and print it next to the claimed cost.
3. Compare the growth. Doubling the input should roughly double an  $O(n \log n)$  algorithm and roughly quadruple an  $O(n^2)$  one.
4. Write down anywhere the measurement and the theory do not agree, and say why.

## COMPLEXITY

The benchmark runs ten algorithms at four sizes. The slowest column is edit distance, and that is what makes the point.

## HOW MY PART CONNECTS TO EVERYBODY ELSE

Every other member quotes a cost on their page. This is the evidence for all of them, in one table.

## LIKELY QUESTIONS

|                                                      |                                                                                                                                                                                                |
|------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Which result is clearest?                            | Edit distance. About 13, 51, 218 and 967 milliseconds at the four sizes. Every time the input doubles, the time goes up about four times. That is exactly what $O(m \times n)$ predicts.       |
| Where does the measurement disagree with the theory? | The Merkle proof, for the reason printed under the table. The theory is right about the proof size. Our code is just slower than it needs to be.                                               |
| Why do the small sizes look noisy?                   | At 100 items several algorithms finish in under a tenth of a millisecond. At that point the timer itself is most of what you are measuring. The growth only becomes clear at the bigger sizes. |
| What is the most expensive part of the ranking?      | Edit distance, by a long way. Everything else is linear or $n \log n$ . If this had to handle real traffic, that is the first thing to replace.                                                |

## SHOW THIS ON SCREEN

Run python benchmark.py and read the table, then the notes printed under it.